CE152: Object-Oriented Programming

2D Graphics - Part 2

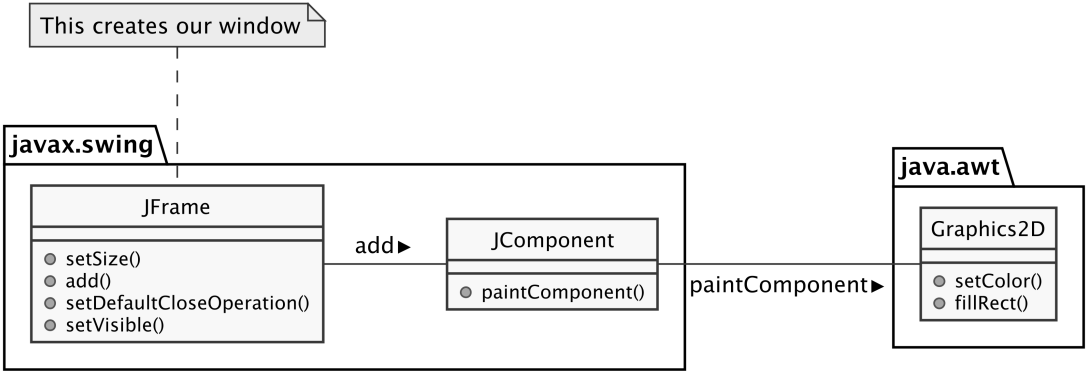
Dr. Sebastian Halder

CSEE

2023

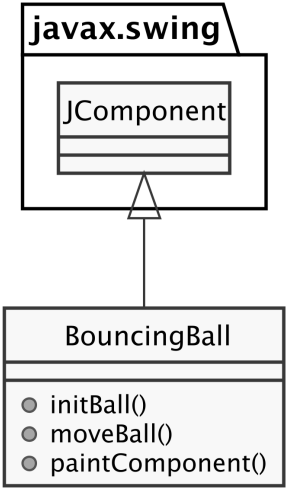
Previous video

Simple 2D Graphics summmary UML



- The connections here do not signal inheritance!

BouncingBall UML



BouncingBall Java (incomplete)

```
public void initBall(){
    ballX = BOX_WIDTH/2;
    ballY = BOX_HEIGHT/2;
    ballVX = (0.5 + Math.random()/2) * MAX_BALL_SPEED;
    ballVY = (0.5 + Math.random()/2) * MAX_BALL_SPEED;
}
public void moveBall() {
    ballX += ballVX;
    if (ballX < BALL_RADIUS || ballX
    + BALL_RADIUS > BOX_WIDTH)
        ballVX *= -1;
    ballY += ballVY;
    if (ballY < BALL_RADIUS || ballY
    + BALL_RADIUS > BOX_HEIGHT)
        ballVY *= -1;
}
```

Continued

```
public void paintComponent(Graphics g) {
    g.setColor(BOX_COLOR);
    g.drawRect(BOX_MARGIN, BOX_MARGIN, BOX_WIDTH,
      BOX_HEIGHT);
    g.setColor(BALL_COLOR);
    int r = BALL_RADIUS;
    g.fillOval( BOX_MARGIN
      + (int)ballX - r, BOX_MARGIN + (int)ballY - r,
            2 * r, 2 * r);
}
```

The while loop in main()

```
animation.initBall();
while (true) {
    animation.moveBall();
    f.repaint();
    Thread.sleep(animation.REPAINT_INTERVAL);
}
```

- This code was *not the best* example of *object-oriented programming (OOP)*

Refining BouncingBall

Issues with the initial design

- We want to write code that is reusable
- In our current design we extended `JComponent` to make our ball
- This makes it easy to add one ball but inefficient to add several
- Generally we want keep things logically separate
- The `JComponent` should be one thing and the shapes we want to animate another
- So what are the core properties of the `Ball`?

Designing the class Ball

Ball
double x double y double vx double vy double radius Color color

- Our ball is defined by the current `x` and `y` coordinates
- Additionally it has a velocity `vx` and `vy`
- The appearance of the ball is defined by the `radius` and the `color`
- We call the variables of the class `Ball`
  - *class variables* if they are static and declared outside a method
  - *instance variables* if they are non-static and declared outside a method
  - *local variables* if they are declared inside a method
- Our ball *must* have a radius and a color so this will be part of the constructor parameters
- Position and velocity will be set randomly

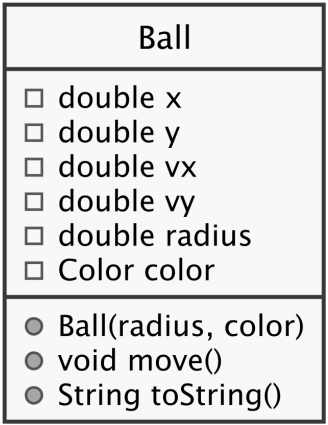
Adding methods

- We will need *methods* to change the state of our ball
- We may also add some methods to display information about the Object to the user
- Clearly we need a method to determine the next position and velocity of the ball within the constraints of the borders of the frame

Adding a meaningful `toString()` method

- All Java classes have the `toString()` method:
  - `public String toString()`
- This method can thus be applied to any Object
- It is automatically applied in cases the JVM decides it is needed:
  - `System.out.println("Ball b1: " + myBall);`
- *Remember*: by default `toString()` returns type and hashcode of the Object:
  - e.g. `Ball@eed1f14`
  - This defines the *identity* of the `Ball`, so that one `Ball` is different from the next
- Therefore we will *override* this method for our class

Ball methods



- Note that in this example the first method is the constructor
- We will have a method `move()` to modify coordinates and velocity
- We will *override* `toString()` to provide meaningful output about `Ball`
- We will define a constructor that sets radius and colour of the ball
  - The constructor is what we invoke with the *new* keyword and is always the class name
  - A class can have multiple constructors with different parameters
- We will also make the variables private and the methods public
  - This is an example of *encapsulation*

Static constants

- There are some aspects of `Ball` that we want to be constant for all instances of `Ball`
- In this case we may choose to make the box in which the ball may move constant
- We can also define a maximum speed
- We declare that they cannot be changed using the keyword `final`
- We will make them `static` so the value can be accessed without instantiating a `Ball` Object

Ball class variables

Ball

- □ double x
- □ double y
- □ double vx
- □ double vy
- □ double radius
- □ Color color
- ○ <u>BOX_WIDTH</u>
- ○ <u>BOX_HEIGHT</u>
- ○ <u>MAX_BALL_SPEED</u>

- ● Ball(radius, color)
- ● void move()
- ● String toString()

- Note the convention to capitalise constants
- Static variables are underlined
- We make these variables public so their values can be retrieved
- We can now create a new `Ball` Object using the following code:

```
Ball b1 = new Ball(50, Color.CYAN);
```

- The variable `b1` will contain a reference to the `Ball` Object (a refresher on that later)
- We can call the methods using variable name dot method name

```
b1.move();
```

Ball class Java

```
import java.awt.Color;
public class Ball {
    // note that (x,y) will be the centre of the ball,
    // while drawings refer to upper-left corner
    // of the bounding box
    private double x, y, vx, vy; // position and velocity
    private int radius;
    private Color color;
    public static final int MAX_BALL_SPEED = 10;
    // dimensions of box in which ball can be moved
    // needed for ball initialisation and ball movement
    public static final int BOX_WIDTH = 600,
      BOX_HEIGHT = 600;
```

Continued

```
// construct ball with random direction starting
// in box center
public Ball(int radius, Color color) {
    x = BOX_WIDTH / 2;
    y = BOX_HEIGHT / 2;
    vx = (2 * Math.random() - 1) * MAX_BALL_SPEED;
    vy = (2 * Math.random() - 1) * MAX_BALL_SPEED;
    this.radius = radius;
    this.color = color;
}
```

Continued

```
public void move() {
    x += vx;
    y += vy;
    if (x < radius || x + radius > BOX_WIDTH)
        vx *= -1;
    if (y < radius || y + radius > BOX_HEIGHT)
        vy *= -1;
}
// we declare that we are overriding the method
@Override
    public String toString() {
        return "Ball(" + x + "," + y + ","
          + vx + "," + vy + ")";
    }
...}
```

Remarks

- Theoretically we could have a variable of type `Ball` in our `Ball` class definition
- This would be called a *recursive data type*
- Note that we are assigning values to all non-final fields in the constructor
- The `move` method modifies coordinates and velocities
  - the velocities `vx` and `vy` are added to the current coordinates `x` and `y`
  - velocities are reversed if `BOX_WIDTH` or `BOX_HEIGHT` are exceeded
  - `radius` is referred to but not modified
- The non-static variables are recreated for each instance of `Ball` (hence the name *instance variables*)
- The static variables once for all instances of the class (hence the name *class variables*)

Keyword this

- Note that we are providing parameters to the constructor with the same name as some of the instance variables
- We use the keyword `this` to ensure we assign them to the instance variables instead of to themselves
- This avoids confusion between the parameter names and the object fields
- We could also use different names for the parameters
  - Using this makes it unnecessary to invent more meaningful variable names

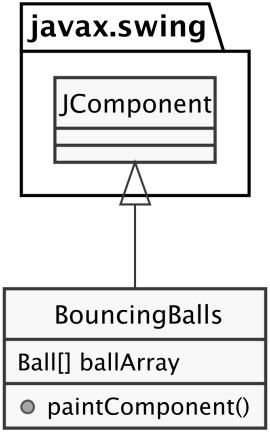
toString()

- When designing `toString()` methods we can aim to include all instance variables of the class
- In this case we chose the coordinates and velocities
  - `"Ball(" + x + "," + y + "," + vx + "," + vy + ")"`
- The goal behind the methods is to be helpful in debugging
- By the way: IDEs such as IntelliJ can autogenerate `toString()` methods
  - Click on Code->Generate and choose `toString()`

Implementing the animation

- We want to have more than one `Ball`
- Thus we will make a JComponent class that can store them in an `Array` of type `Ball` and create the `Graphics` in the `paintComponent()` method
- In the `main()` method we will loop through the array and modify the coordinates

BouncingBalls UML



BouncingBalls Java - Constructor

```java
public final Ball[] ballArray = new Ball[10];

public BouncingBalls() {
    Random r = new Random();
    for (int i = 0; i < ballArray.length; i++) {
        Color c = new Color(r.nextInt(255),
          r.nextInt(255), r.nextInt(255));
        int size = (int) ((1
          + r.nextDouble()) * BALL_RADIUS);
        ballArray[i] = new Ball(size, c);
    }
}
```

BouncingBalls Java - paintComponent

```
public void paintComponent(Graphics g) {
    g.setColor(BOX_COLOR);
    g.drawRect(BOX_MARGIN, BOX_MARGIN,
      Ball.BOX_WIDTH, Ball.BOX_HEIGHT);
    for (Ball b : ballArray) {
        g.setColor(b.getColor());
        int r = b.getRadius();
        g.fillOval(BOX_MARGIN + (int) b.getX()
          - r, BOX_MARGIN + (int) b.getY()
          - r, 2 * r, 2 * r);
    }
}
```

main()

```
BouncingBalls animation = new BouncingBalls();
JFrame f = new JFrame("Bouncing Ball Animation");
f.getContentPane().setPreferredSize(new Dimension(Ball.BOX_WIDTH + 2
          * animation.BOX_MARGIN,
          Ball.BOX_HEIGHT + 2 * animation.BOX_MARGIN));
f.add(animation);
f.setResizable(false);
f.pack(); // resizes to at least preferred size
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
f.setVisible(true);
while (true) {
    Thread.sleep(animation.REPAINT_INTERVAL);
    for (Ball b : animation.ballArray) {
        b.move();}
    f.repaint();}}
```

Let's have a look

- Run the example
- Add more balls

Further remarks

- Remember that when we create a new Object such as `Ball b1 = new Ball(50, Color.RED);` and then assign this variable to another such as `Ball b2 = b1` then b2 will contain a reference to the same Object as `b1`
- Thus `b1==b2` will be `true`
- If instead we write `Ball b2 = new Ball(50, Color.RED);` (the parameters are the same) and `b1==b2` be `false`
- If the reasons for this are unclear please rewatch the video on conditionals, ask me during the lecture, labs or visit the support class

Reminder on Garbage collection

- Objects are stored on the heap
- If Objects become inaccessible they will be removed by the garbage collector
- For example:

```
Ball b1 =  new Ball(50, Color.RED);
Ball b2 =  new Ball(40, Color.BLUE);
b1 = b2;
```

- Now `b1` and `b2` will refer to the same Object
- The original `Ball` represented by the variable `b1` has become inaccessible and will be removed via garbage collection

Objects as arguments

- In the same context it is important to remember that Objects are passed as a reference not as a value
- Thus if we pass a `Ball` Object as a parameter and call `move()` within the method the original object will also be moved
- Also if we put the same `Ball` Object in different places in the `Ball[] ballArray` these will all be references to the same Object
- If we want to recreate the same `Ball` twice we have to implement a `copy()` method

Copy method Java

```java
public class Ball {...
    public Ball copy() {
        return new Ball(x, y, vx, vy, radius, color);
    }
}
// Test
Ball b1 = new Ball(20, Color.RED);
Ball b2 = b1.copy();
System.out.println("b1 = " + b1);
System.out.println("b2 = " + b2);
}
```

```
// Program output
b1 = Ball(300.0,300.0,-7.812954048260046,-6.52089160400974)
b2 = Ball(300.0,300.0,-7.812954048260046,-6.52089160400974)
```

null

- Recall the distinction between primitive data types and Objects
- Objects are treated as *references* and are thus also called *reference types*
- *Reference types* can be empty or *null*
  - This means they currently do not refer to an Object
- We can test if a variable *references* an Object using comparison operators to check if the variable has been assigned a value

```
(x == null), (x != null)
```

- Methods can also return *null* if the Object they would normally return cannot be returned
  - This can be an alternative to throwing an exception

Encapsulation

- Take a moment to consider the principle of encapsulation in the `Ball` class
- Coordinates and velocities are private
- We cannot access them directly
- We also do not have to worry about modifying them to move the `Ball`
- The method `move()` does it for us
- Access to the variables is `encapsulated` and we don't have to worry that someone might write code depending on these variables
- We can safely exchange them and calculate `move()` in another way

Defining encapsulation

- Recall that we defined access to the members of a class using the following keywords:
  - `public`: access open to all (circle in UML)
  - `protected`: access only to subclasses (diamond in UML)
  - `private`: hidden to the outside of class (square in UML)
- If we do not declare an access level all classes in the same package can access
- `protected` is not frequently used in Java

Concurrency

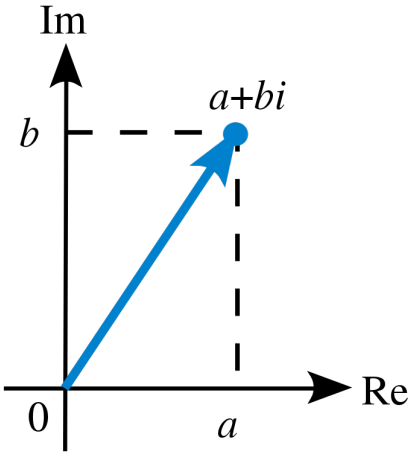
- In our main method we actually started two threads of control
  - `main` updates the ball object and requesting paint refreshes in an infinite loop
  - The Swing GUI (`JFrame`) has its own thread of control and handles all the user interactions and also the `repaint()` calls from our loop
- This means multiple computations are happening at the same time and is called *concurrency*
- In more complicated programs synchronisation would be needed to ensure the correct behaviour
- By the way, `Thread.sleep()` throws `InterruptedException` that we must handle or declare
- We chose to declare it; *how would it be handled?*
- If unclear please watch video on exceptions

Complex numbers and Mandelbrot sets

Complex numbers

- ***The mathematical aspects of complex numbers and Mandelbrot sets will not be part of the exam***
- Complex numbers have the form $a + bi$ where a and b are real (normal) numbers and i is the "imaginary unit"
- They were invented in the 16th century and allow solving science and engineering equations that could otherwise not be solved e.g.:
  - $f(x) = 5 + \sqrt{-x}$ is $5 + i\sqrt{x}$
- You may also think of them as 2D vectors

Complex numbers as 2D vectors



- Image from Wikipedia

Complex number class

- We want to create a Java class that represents these numbers
- We need to instance variables of type `double` that represent the real and imaginary parts of the complex numbers
- We want our complex numbers to be *immutable*
  - That means the content cannot be modified
  - We will ensure this by making the real and imaginary parts `final`

Complex number class Java

```
public class Complex {
    public final double re, im;

    public Complex (double re, double im) {
        this.re = re;
        this.im = im;
    }

    public Complex (double re) {
        this.re = re;
        this.im = 0.0;
    }

    public Complex conj () {
        return new Complex (re, -im);
    }
```

Continued

```
    public Complex add (Complex other) {
        return new Complex
                (re + other.re, im + other.im);
    }

    public Complex minus (Complex other) {
        return new Complex
                (re - other.re, im - other.im);
    }

    public Complex mult (Complex other) {
        return new Complex
                (re * other.re - im* other.im,
                        re * other.im + im * other.re);
    }
```

Continued

```
public Complex reciprocal() {
    double d = re * re + im * im;
    return new Complex (re / d, -im/d);}

public Complex div(Complex other) {
    return this.mult(other.reciprocal());}

public double abs() {
    return Math.sqrt(re*re + im*im);}

public double arg() {
    return Math.atan2(im, re);}

public String toString() {
    return ("(" + re + " + " + im + "i)");
  }}
```

Behaviour of the complex numbers

```
Complex c1 = new Complex (1,1);
Complex c2 = new Complex (2,1);

c2.conj()          //  (2.0 + -1.0i)
c2.abs()           //  2.23606797749979
c2.arg()           //  0.7853981633974483
c2.reciprocal()    //  (0.4 + -0.2i)
c1.add(c2)         //  (3.0 + 2.0i)
c1.minus(c2)       //  (-1.0 + 0.0i)
c1.mult(c2)        //  (1.0 + 3.0i)
c1.div(c2)         //  (0.6000000000000001 + 0.2i)
```

The Mandelbrot set

- We want to use these complex number to approximate the Mandelbrot set of a given complex number c
- Every complex number is either part of the Mandelbrot set or not
- A complex number c is said to be part of the Mandelbrot M set if the sequence of numbers starting with $z_0 = 0$ is bounded

$$z_{n+1} = z_n^2 + c,$$

$$c \in M \leftrightarrow \limsup_{n \to \infty} |z_{n+1}| \leq 2$$

- For example this sequence is ***not bounded for 1*** and ***bounded for -1***... therefore $c = -1$ would be part of the set M
- Have a look at https://en.wikipedia.org/wiki/Mandelbrot_set for more information and some animations if you are interested

How to visualise this

- We have to determine if the sequence exceeds 2 towards infinity
- Obviously we cannot calculate this
- Therefore we look at the first 255 steps only
- We also cannot look at every complex number
- Therefore we only sample an N-by-N grid of numbers

The Mandelbrot method

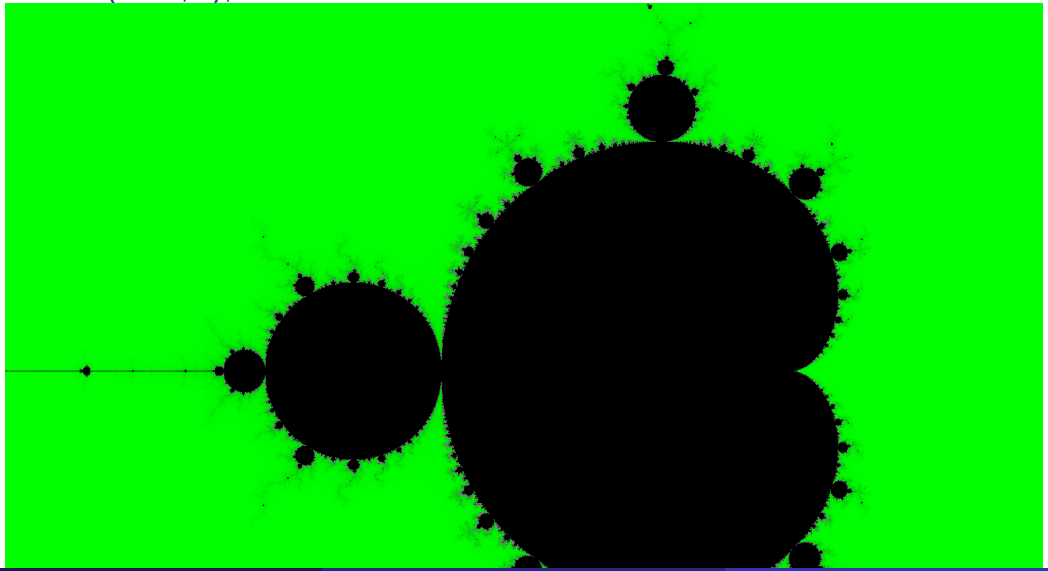
```
// checks if the given number is part of the
// Mandelbrot set
public static Color mand(Complex z0) {
    Complex z = z0;
    for (int t = 0; t < 255; t++) {
        if (z.abs() > 2.0)
            return new Color(t, 255 - t, t);
        z = z.mult(z);
        z = z.add(z0);
    }
    return Color.BLACK;
}
```

- How would we change the background colour?

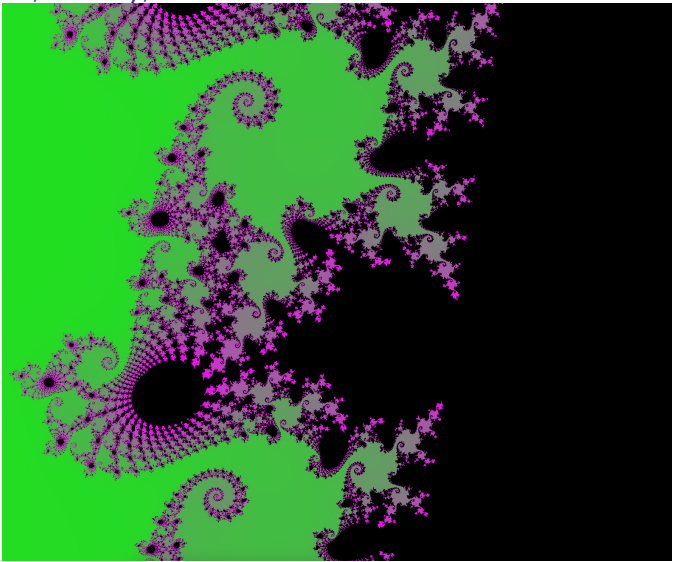
Checking a set of complex numbers

```
// Iterates over NxN numbers centered around cx/cy
// with an extent of size (the smaller the more we
// zoom in)
public Mandelbrot(double cx, double cy, double size) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            double x0 = cx - size / 2 + size * i / N;
            double y0 = cy - size / 2 + size * j / N;
            Complex z0 = new Complex(x0, y0);
            Color color = mand(z0);
            im.setRGB(i, N - 1 - j, color.getRGB());
        }
    }
}
```

Result for (-0.5,0), size=3



Result for (-0.745,0.095), size=0.01



Summary

Designing for OOP

- Try to break down your program into classes
- Aim for reusability by ensuring:
  - high cohesion: data and methods in one class are closely related
  - low coupling: classes should not depend on one another unless necessary
- In practice there will always be compromises
- E.g. using standard API classes will create dependencies but is normally "a good thing"

Thank you and see you in the next video!